Nama: Zahwa Nafiza Azzahra

NIM: H1D024015

Shift KRS: H

Shift Sekarang: F

1. Pendahuluan

Program ini adalah Sistem Pakar (Expert System) berbasis Python yang dirancang untuk membantu pengguna mengidentifikasi kemungkinan penyakit pada area Telinga, Hidung, dan Tenggorokan (THT). Program menggunakan metode Forward Chaining untuk mencocokkan gejala yang dialami pengguna dengan basis pengetahuan yang telah tersimpan, kemudian menampilkan hasil diagnosa beserta saran penanganan.

Teknologi yang Digunakan
Bahasa Pemrograman : Python 3Framework GUI : Tkinter (built-in Python)Metode Inferensi : Forward ChainingBasis Pengetahuan : 37 gejala, 23 penyakit THT

2. Arsitektur Program

Kode terbagi menjadi tiga bagian utama yang terpisah secara logis, mengikuti pola arsitektur yang bersih:

Bagian	Nama	Fungsi
1	KNOWLEDGE BASE	Menyimpan data gejala, penyakit, deskripsi, dan saran.
2	INFERENCE ENGINE	Fungsi diagnosa() — mencocokkan gejala dengan forward chaining.
3	GUI (Tkinter)	Antarmuka grafis untuk interaksi pengguna (kelas App).

3. Basis Pengetahuan (Knowledge Base)

Basis pengetahuan adalah komponen inti sistem pakar. Seluruh data tersimpan sebagai dictionary Python (struktur key-value) yang mudah diperbarui tanpa mengubah logika inferensi.

3.1 Kamus Gejala — GEJALA

Dictionary GEJALA berisi 37 gejala dengan kode unik (G1–G37). Kode digunakan sebagai referensi internal sehingga nama gejala dapat diubah tanpa mempengaruhi relasi penyakit.

```
GEJALA = {
    "G1":  "Nafas abnormal",
    "G2":  "Suara serak",
    "G3":  "Perubahan kulit",
    ...
    "G37": "Demam",
}
```

Setiap kode gejala kemudian diubah menjadi kalimat pertanyaan secara otomatis melalui dictionary comprehension:

```
PERTANYAAN = {
    kode: f"Apakah Anda mengalami {deskripsi.lower()}?"
    for kode, deskripsi in GEJALA.items()
}
```

Teknik ini memastikan pertanyaan selalu sinkron dengan daftar gejala — jika gejala diubah, pertanyaannya ikut berubah secara otomatis.

3.2 Peta Penyakit — PENYAKIT

Dictionary PENYAKIT memetakan nama penyakit ke daftar kode gejala yang menjadi indikatornya. Inilah aturan (rules) dalam sistem pakar ini.

```
PENYAKIT = {
    "Tonsilitis":          ["G37","G12","G5","G27","G6","G21"],
    "Sinusitis Maksilaris":["G37","G12","G27","G17","G33","G36","G29"],
    "Sinusitis Frontalis":
["G37","G12","G27","G17","G33","G36","G21","G26"],
    ...
    "Vertigo Postural":    ["G24"],
}
```

Jumlah gejala per penyakit bervariasi (1 sampai 10 gejala). Penyakit dengan sedikit gejala (seperti Vertigo Postural: hanya G24) lebih mudah mencapai ambang batas 40%, sementara penyakit dengan banyak gejala memerlukan lebih banyak kecocokan.

3.3 Deskripsi & Saran Penanganan

Dua dictionary tambahan menyediakan informasi yang ditampilkan kepada pengguna setelah diagnosa berhasil:

- DESKRIPSI_PENYAKIT — Penjelasan singkat tentang kondisi medis tersebut.
- SARAN_PENYAKIT — Rekomendasi langkah penanganan awal untuk setiap penyakit.

Contoh untuk penyakit Tonsilitis:

```
"Tonsilitis": "Peradangan pada amandel (tonsil) yang biasanya
              disebabkan oleh infeksi bakteri atau virus."
```

```
"Tonsilitis": "Konsultasikan ke dokter THT. Pengobatan meliputi
              antibiotik dan pereda nyeri. Jika sering kambuh,
              tonsilektomi mungkin direkomendasikan."
```

4. Mesin Inferensi (Inference Engine)

Mesin inferensi adalah jantung sistem pakar. Fungsi diagnosa() mengimplementasikan metode Forward Chaining — dimulai dari fakta (gejala yang diakui pengguna), kemudian mencocokkan dengan aturan untuk menghasilkan kesimpulan (diagnosis).

4.1 Algoritma Forward Chaining

Langkah-langkah algoritma:

- Terima input: daftar kode gejala positif (yang diakui pengguna).
- Iterasi setiap penyakit dalam PENYAKIT.
- Hitung irisan antara gejala penyakit dan gejala positif pengguna.
- Hitung persentase kecocokan = (gejala cocok / total gejala penyakit) × 100%.
- Jika persentase >= 40%, masukkan penyakit ke daftar hasil.
- Urutkan hasil dari persentase tertinggi ke terendah.

4.2 Kode Fungsi diagnosa()

```
def diagnosa(gejala_positif):
    hasil = []
    for nama, daftar_gejala in PENYAKIT.items():
        # Hitung gejala yang cocok (irisan dua himpunan)
        cocok = set(daftar_gejala) & set(gejala_positif)
        if len(cocok) == 0:
            continue
        # Persentase kecocokan
        persen = round(len(cocok) / len(daftar_gejala) * 100)
        if persen >= 40:  # Ambang batas minimal
            hasil.append({
                "nama": nama,
                "skor": persen,
                "deskripsi": DESKRIPSI_PENYAKIT.get(nama, "-"),
                "solusi": SARAN_PENYAKIT.get(nama, "-"),
            })
    hasil.sort(key=lambda x: x["skor"], reverse=True)
    return hasil
```

4.3 Penjelasan Detail Baris per Baris

set(daftar_gejala) & set(gejala_positif)
Operator & pada Python set menghasilkan irisan — elemen yang ada di kedua himpunan. Contoh: {'G1','G5','G6'} & {'G5','G6','G37'} = {'G5','G6'} → 2 gejala cocok.

Ambang Batas 40%
Nilai 40% dipilih sebagai batas minimal agar hasil tidak terlalu ketat (tidak ada diagnosis) maupun terlalu longgar (terlalu banyak diagnosis tidak relevan). Nilai ini dapat disesuaikan sesuai kebutuhan klinis.

hasil.sort(key=lambda x: x['skor'], reverse=True)

Mengurutkan hasil dari skor tertinggi ke terendah menggunakan fungsi lambda sebagai key. Dengan reverse=True, penyakit dengan kecocokan terbesar muncul pertama.

5. Antarmuka Grafis (GUI) — Kelas App

Antarmuka pengguna dibangun menggunakan Tkinter, library GUI bawaan Python. Seluruh logika tampilan dikapsulasi dalam kelas App yang mengikuti pola Object-Oriented Programming (OOP).

5.1 Inisialisasi — __init__()

```
def __init__(self, root):
    self.root = root
    self.root.title("Sistem Pakar Diagnosa Penyakit THT")
    self.root.resizable(False, False)
    self.gejala_list = list(PERTANYAAN.keys())
    self.gejala_pos = []   # gejala yang diakui
    self.gejala_neg = []   # gejala yang ditolak
    self.index = 0         # indeks pertanyaan saat ini
    self._build_ui()
    self._set_state_awal()
```

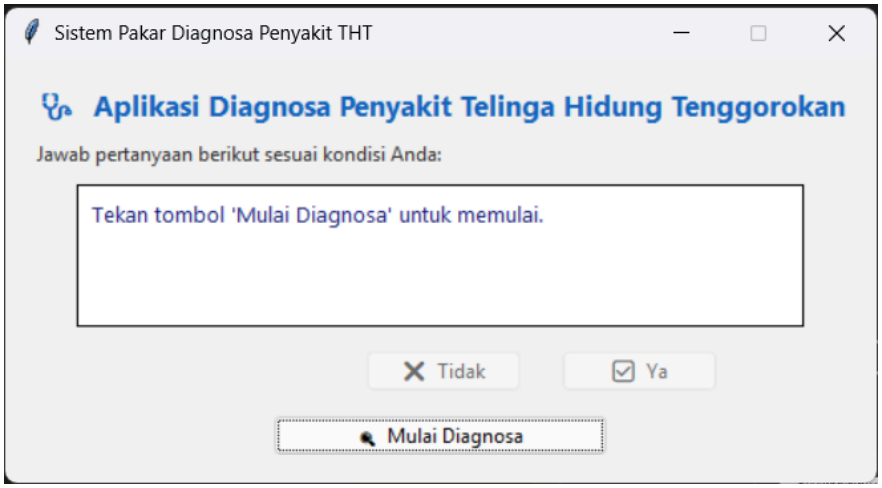
Variabel instance penting:

Variabel	Keterangan
gejala_list	Daftar kode G1–G37 dalam urutan iterasi.
gejala_pos	Kode gejala yang dijawab 'Ya' oleh pengguna.
gejala_neg	Kode gejala yang dijawab 'Tidak' (untuk referensi).
index	Penunjuk pertanyaan yang sedang ditampilkan.

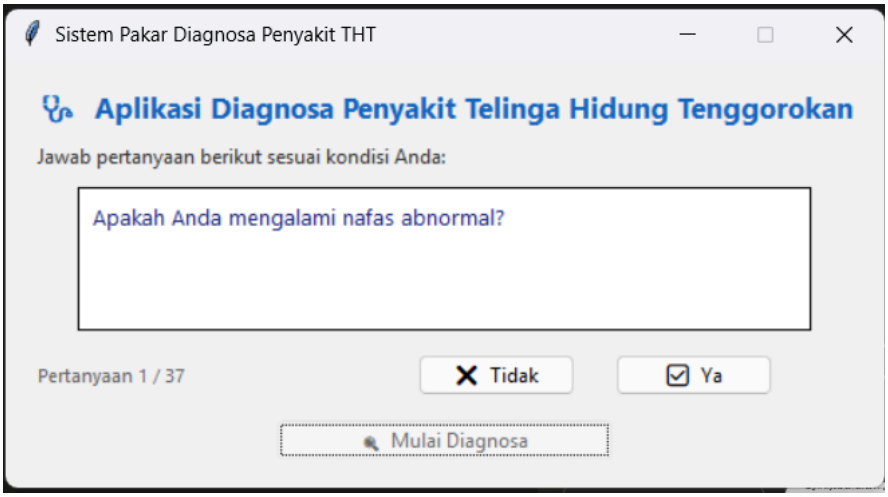
5.2 Pembangunan UI — _build_ui()

Metode _build_ui() mendefinisikan semua widget Tkinter menggunakan Grid Layout Manager. Komponen UI terdiri dari:

- Label Judul — Menampilkan nama aplikasi dengan font biru tebal.
- Label Subjudul — Instruksi singkat untuk pengguna.
- Widget Text (kotak) — Menampilkan teks pertanyaan, read-only (state='disabled').
- Label Progress — Menampilkan nomor pertanyaan saat ini (misalnya 'Pertanyaan 5 / 37').
- Tombol Tidak (btn_tidak) — Merekam jawaban negatif.
- Tombol Ya (btn_ya) — Merekam jawaban positif.
- Tombol Mulai / Ulangi (btn_mulai) — Memulai atau mengulang sesi diagnosa.



Gambar 1 — Tampilan awal aplikasi saat pertama kali dibuka



Gambar 2 — Tampilan saat sesi diagnosa berjalan (Pertanyaan 1 dari 37)

5.3 Alur Sesi Diagnosa

1. Pengguna klik [Mulai Diagnosa] → _mulai() dipanggil
2. Variabel gejala_pos, gejala_neg, index direset ke 0
3. _tanya_berikutnya() menampilkan pertanyaan gejala index ke-N
4. Pengguna klik [Ya] atau [Tidak] → _jawab(True/False) dipanggil
5. Kode gejala dimasukkan ke gejala_pos atau gejala_neg
6. index++ → kembali ke langkah 3 sampai semua 37 gejala ditanyakan
7. _selesai() memanggil diagnosa() dan menampilkan hasil

5.4 Metode _jawab() & _tanya_berikutnya()

```
def _jawab(self, jawaban):
    kode_gejala = self.gejala_list[self.index]
    if jawaban:
        self.gejala_pos.append(kode_gejala)
    else:
```

```
        self.gejala_neg.append(kode_gejala)
    self.index += 1
    self._tanya_berikutnya()
```

```
def _tanya_berikutnya(self):
    if self.index >= len(self.gejala_list):
        self._selesai()   # semua pertanyaan habis
        return
    kode = self.gejala_list[self.index]
    self._tampilkan_teks(PERTANYAAN[kode])
    self.progress_label.configure(
        text=f"Pertanyaan {self.index+1} / {len(self.gejala_list)}")
```

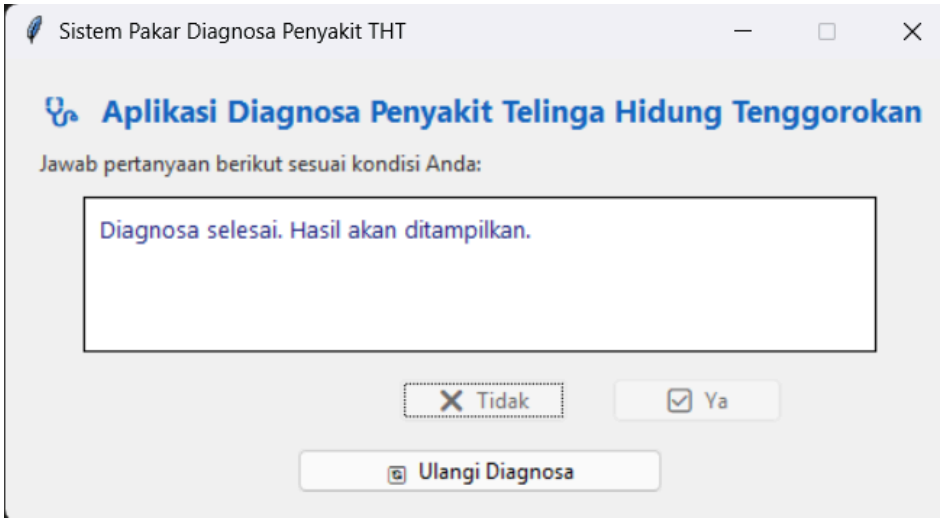
5.5 Menampilkan Hasil — _selesai()

Setelah semua pertanyaan terjawab, _selesai() memanggil fungsi diagnosa() dan memformat hasilnya ke dalam messagebox. Setiap penyakit yang ditemukan ditampilkan beserta persentase kecocokan, deskripsi, dan saran penanganan.

```
def _selesai(self):
    hasil = diagnosa(self.gejala_pos)
    if not hasil:
        messagebox.showinfo("Hasil", "Tidak ada penyakit yang
cocok...")
    else:
        pesan = ""
        for i, item in enumerate(hasil, 1):
            pesan += f"[{i}] {item['nama']}\n"
            pesan += f"    Tingkat Kecocokan: {item['skor']}%\n"
            pesan += f"    Deskripsi: {item['deskripsi']}\n"
            pesan += f"    Saran: {item['solusi']}\n\n"
        pesan += "DISCLAIMER: Bukan pengganti konsultasi dokter."
        messagebox.showinfo("Hasil Diagnosa", pesan)
```

6. Contoh Hasil Diagnosa

Berikut adalah tampilan aplikasi setelah sesi diagnosa selesai dijalankan:



Gambar 3 — Pesan konfirmasi setelah seluruh 37 pertanyaan dijawab

Gambar 4 — Jendela Hasil Diagnosa menampilkan daftar penyakit beserta tingkat kecocokan, deskripsi, dan saran penanganan

Pada contoh di atas, sistem berhasil mengidentifikasi 6 kemungkinan penyakit:

- [1] Barotitis Media — 50% kecocokan
- [2] Kanker Nasofaring — 50% kecocokan

- [3] Kanker Tonsil — 50% kecocokan
- [4] Neuronitis Vestibularis — 50% kecocokan
- [5] Otosklerosis — 50% kecocokan
- [6] Sinusitis Maksilaris — 43% kecocokan

Semua hasil diurutkan dari persentase tertinggi ke terendah, dan di bagian bawah selalu muncul peringatan bahwa hasil ini bukan pengganti konsultasi dokter profesional.

7. Entry Point Program

Blok if __name__ == '__main__' adalah titik masuk program Python. Ini memastikan kode hanya dijalankan saat file dieksekusi langsung.

```
if __name__ == "__main__":
    root = tk.Tk()    # Buat jendela utama Tkinter
    App(root)         # Inisialisasi kelas App
    root.mainloop()   # Mulai event loop GUI
```